

Lesson 01: Hello, World

In this lesson, you will build and run a tiny C program on a Linux laptop. You will first make the supplied program work. Then you will change one greeting and see what happens.

Learning Goals

By the end of this lesson, you will be able to:

- describe **source code** as program text a person can read and edit;
- **build** source code into a program the laptop can run;
- **run** that program from the terminal; and
- identify a printed greeting as the program's **output**.

Before You Begin

You need a teacher-prepared Linux laptop with VS Code and the GCC C compiler. You do not need an Arduino or any electronic parts for this lesson.

Ask your teacher to confirm that GCC is ready. Installing programs or changing computer permissions is a teacher task.

Core Mission: Make the Laptop Say Hello

Goal: Build and run the supplied program so you can see its greeting in the terminal.

Prepare: Open VS Code. Do not change `main.c` yet; first run the supplied program exactly as it is.

1. Choose **File > Open Folder...** and select the `coding-fundamentals-lessons/01-hello-world` folder. Open the lesson folder, not only the `main.c` file.
2. Choose **Terminal > New Terminal**.
3. Run this command to print the terminal's current folder:

```
pwd
```

Check that the path ends with `coding-fundamentals-lessons/01-hello-world`.

4. Run this command to list the lesson files:

```
ls
```

You should see `main.c` and `README.md`. You might also see `hello-world` if the program was built before. If you do not see `main.c`, stop and reopen the correct folder.

5. Build the supplied source code with this exact command:

```
gcc -std=c11 -Wall -Wextra -Werror main.c -o hello-world
```

A successful build usually prints no message and returns to the prompt.

6. Run the program:

```
./hello-world
```

Notice: What exact message appeared?

Actual output: _____

Did the terminal place its next prompt on a new line? **Yes / No**

Stop here with the terminal open. You have completed the Core Mission. Read the next section before changing the greeting.

Name What Just Happened

You used five connected parts:

source code	build	executable	run
output main.c world!	—with GCC—>	hello-world	—————> Hello,

- A **program** is a group of instructions for a computer.
- **Source code** is program text that a person can read and edit. This lesson's source file is `main.c`.
- To **build** means to turn source code into a form the laptop can run. GCC built the executable named `hello-world`.
- To **run** means to start the program and let it follow its instructions. The `./` in `./hello-world` says to run the program from this folder.
- **Output** is information produced by a program. The greeting you saw is output.

A successful build and a successful run are different events. Building makes the executable. Running it lets you observe its behavior.

Find the Focus Instruction

Open `main.c`. It contains:

```
#include <stdio.h>
```

```
int main(void) {  
    printf("Hello, world!\n");  
    return 0;  
}
```

For this lesson, focus on this instruction:

```
printf("Hello, world!\n");
```

printf produces text output. The words inside the quotation marks are the message. The **\n** moves the terminal to a new line after the message.

The other lines help make this a complete C program. You do not need to memorize them or explain all the punctuation in this lesson.

Try One Change: Write a New Greeting

Change only the greeting inside the quotation marks. Keep **\n** at the end so the terminal moves to a new line. For example:

```
printf("Hello, robot builder!\n");
```

1. Write the exact new message you chose:

New message: _____

2. **Predict:** What exact output will appear after you save, rebuild, and run?

Prediction: _____

3. Edit the **printf** instruction in **main.c**. Change only the message inside its quotation marks, keep **\n**, and save the file.

4. Rebuild the executable:

```
gcc -std=c11 -Wall -Wextra -Werror main.c -o hello-world
```

5. Run the newly built program:

```
./hello-world
```

6. Record what appeared:

Actual output: _____

7. **Explain:** Complete this sentence:

I changed _____ in the source code. After I rebuilt and ran the program, the output changed from _____ to _____.

Restore or Compare

Choose one:

- Restore the original instruction, save, rebuild, and confirm that the output is **Hello, world!** again.

```
printf("Hello, world!\n");
```

- Keep your new greeting and point to the exact words in **main.c** that caused each changed word in the output. Say what stayed the same between the original and new runs.

Optional Cleanup and Rebuild

The executable was generated from **main.c**, so it can be removed and built again. To remove only the generated executable, run:

```
rm hello-world
```

Run **ls** and confirm that **main.c** remains. Then repeat the GCC build command and run the program again. Never add **main.c** to the **rm** command.

Troubleshooting

- **pwd ends with a different folder:** Use **File > Open Folder...** and select **01-hello-world**, then open a new terminal.
- **ls does not show main.c:** You are in the wrong folder. Do not build until **main.c** appears.
- **The terminal says gcc: command not found:** Stop and ask the teacher to prepare GCC. This is not a source-code problem.
- **GCC prints an error after your edit:** Compare your **printf** instruction with the example. Check its two quotation marks, closing parenthesis, semicolon, and **\n**, then ask your teacher to help read the first error.
- **The build returns silently:** That is normal. Run the program next.
- **The terminal says ./hello-world: No such file or directory:** The program was not built in this folder. Check **pwd** and **ls**, then run the GCC command again.
- **Your old greeting still appears:** Save **main.c**, rebuild, and then run the program again.
- **The words appear with \n printed at the end:** Check that the message contains one backslash followed by **n**, without an extra backslash.

When your observation and explanation are complete, continue to [lesson 02: Name Greeting](#).